

UT.6.3 PROGRAMACIÓN EN BASES DE DATOS: PROCEDIMIENTOS Y FUNCIONES

1. INTRODUCCIÓN	2
2. PROCEDIMIENTOS	2
2.1 LLAMADAS A PROCEDIMIENTOS	3
2.1.1 PARÁMETROS DE LOS SUBPROGRAMAS	4
2.2 RECOMPILACIÓN Y ELIMINACIÓN DE PROCEDIMIENTOS	5
3. FUNCIONES	6
3.1 LLAMADAS A FUNCIONES	7
3.2 BORRADO DE FUNCIONES	8
4. PAQUETES (PACKAGE)	8
4.1 VENTAJAS DEL USO DE PAQUETES	9

1.INTRODUCCIÓN

Los **subprogramas** son bloques de código PL/SQL, **referenciados bajo un nombre**, que realizan una acción determinada. Les podemos pasar parámetros y los podemos invocar.

Además, los subprogramas pueden estar almacenados en la base de datos o estar encerrados en otros bloques. Si el programa está almacenado en la base de datos, podremos invocarlo si tenemos permisos suficientes y si está encerrado en otro bloque lo podremos invocar si tenemos visibilidad sobre el mismo.

Hay **dos clases de subprogramas: las funciones y los procedimientos**. Las **funciones devuelven un valor y los procedimientos no**.

2.PROCEDIMIENTOS

Un **procedimiento** es un bloque PL/SQL nombrado que realiza una determinada acción. Está almacenado en la base de datos como un objeto de la misma, y puede ser ejecutado múltiples veces, por lo que permiten la reutilización, ya que una vez validados pueden utilizarse en diferentes aplicaciones. Si necesitamos cambiar algo en la definición del procedimiento, solamente se verá afectado el código del mismo, lo que simplifica a su vez las tareas de mantenimiento.

Sintaxis:

```
CREATE [OR REPLACE] PROCEDURE nombre [( parametro [, parametro] ...
)] IS|AS
[declaraciones_locales]
BEGIN
sentencias_ejecutables
[EXCEPTION manejadores_de_excepciones]
END [nombre];
```

Donde:

```
parametro := nombre_parametro [IN|OUT|IN OUT] tipo_dato
[{:|=|DEFAULT} expresion]
```

Ejemplo:

```
CREATE OR REPLACE PROCEDURE PROC1 (p_ID IN empleado.codemp%TYPE) IS
BEGIN
    UPDATE empleado SET salemp = salemp * 1.10 WHERE codemp = p_ID;
END;
/
```

Este procedimiento subirá un 10% el salario del empleado cuyo identificador reciba como parámetro en la llamada.

Si especificamos OR REPLACE y el subprograma ya existía, éste será reemplazado. Si no lo especificamos y el subprograma ya existe, Oracle nos devolverá un error indicando que el nombre ya está siendo utilizado por otro objeto de la base de datos.

El comando SQLPlus show errors tras la compilación de un procedimiento o función nos muestra los errores si los hay.

2.1 LLAMADAS A PROCEDIMIENTOS

En general un procedimiento se hace para ser llamado desde otro programa (bien de la propia BBDD o bien externo).

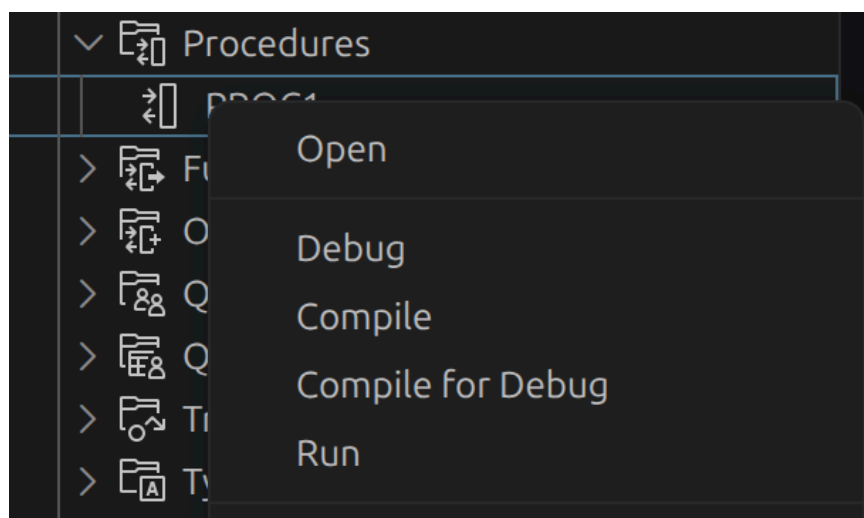
Para llamarlo desde otro programa de la BBDD simplemente hay que poner el nombre del procedimiento, pasarle los parámetros y ya está. Lo veremos más adelante con ejemplos.

Para llamarlo desde un cliente de BBDD y poder hacer pruebas hay varias formas. En general se hará un CALL o un EXECUTE del procedimiento, aunque el cliente de BBDD seguro que ofrece una forma de ejecutarlo. Lo vemos con la extensión de SQL Developer para Visual Studio Code:

```
EXECUTE PROC1 (7) ;  
CALL PROC1 (7) ;
```

Aunque son similares, como regla general usad **EXECUTE** cuando estéis trabajando en la consola de SQL Developer o SQL*Plus, ya que es más flexible y permite usar tipos de datos de PL/SQL.

Usad **CALL** cuando necesitéis que vuestra llamada sea una sentencia SQL pura (por ejemplo, si la llamáis desde una aplicación Java o Python) o cuando queráis seguir el estándar ANSI.



2.1.1 PARÁMETROS DE LOS SUBPROGRAMAS

Las variables pasadas como parámetros a un subprograma son llamadas **parámetros actuales**. Las variables referenciadas en la especificación del subprograma como parámetros, son llamadas **parámetros formales**.

Los parámetros pueden ser de **entrada** al subprograma, de **salida**, o de **entrada/salida**. **Por defecto** si a un parámetro no le especificamos el modo, éste **será de entrada**. **Si el parámetro es de salida o de entrada/salida, el parámetro actual debe ser una variable.**

Un **parámetro de entrada** permite que pasemos valores al subprograma y **no puede ser modificado** en el cuerpo del subprograma. El parámetro actual pasado a un subprograma como parámetro formal de entrada puede ser una constante o una variable.

Un **parámetro de salida** permite **devolver valores** y dentro del subprograma actúa como variable no inicializada. El parámetro formal debe ser siempre una variable.

Un **parámetro de entrada-salida** se utiliza para **pasar valores al subprograma y/o para recibirlos**, por lo que un parámetro formal que actúe como parámetro actual de entrada-salida siempre deberá ser una variable.

Los parámetros de entrada los podemos inicializar a un valor por defecto. Si un subprograma tiene un parámetro inicializado con un valor por defecto, podemos invocarlo prescindiendo del parámetro y aceptando el valor por defecto o pasando el parámetro y sobrescribiendo el valor por defecto.

IN	OUT	IN OUT
Por defecto.	Tiene que especificarse.	Tiene que especificarse.
Valor que se pasa al subprograma.	Devuelve al entorno de llamada.	Valor que se pasa al subprograma. Devuelve al entorno de llamada.

Cuando llamamos a un subprograma, los parámetros actuales podemos escribirlos utilizando notación posicional o notación nombrada, es decir, la asociación entre parámetros actuales y formales podemos hacerla por posición o por nombre:

- **Posicional:** Lista de valores en el orden en el que se declaran los parámetros.
- **Asociación nombrada:** Lista de valores en orden arbitrario mediante la asignación =>
- **Mixto:** Mezcla de los dos anteriores. Lista de los primeros valores posicionalmente, y el resto utilizando sintaxis especial nombrada.

Ejemplos:

Procedimiento con parámetros de entrada (IN) y de salida (OUT):

```

CREATE OR REPLACE PROCEDURE PROC2 (p_ID IN empleado.codemp%TYPE,
p_sal OUT empleado.salemp%TYPE,
p_dep OUT empleado.coddep%TYPE) IS
BEGIN
    SELECT salemp, coddep INTO p_sal, p_dep
    FROM empleado
    WHERE codemp = p_ID;
END;
/

```

Veamos ahora cómo sería una llamada a este procedimiento desde otro procedimiento (en este caso anónimo), con paso posicional de parámetros:

```

DECLARE
    p_salario empleado.salemp%TYPE;
    p_departamento empleado.coddep%TYPE;
    p_cod empleado.codemp%TYPE;
BEGIN
    p_cod := 7;
    PROC2(p_cod, p_salario, p_departamento);
    DBMS_OUTPUT.PUT_LINE('Salario: ' || p_salario);
    DBMS_OUTPUT.PUT_LINE('Departamento: ' || p_departamento);
END;
/

```

La misma llamada con asociación nombrada de parámetros podría ser:

```

DECLARE
    p_salario empleado.salemp%TYPE;
    p_departamento empleado.coddep%TYPE;
    p_cod empleado.codemp%TYPE;
BEGIN
    p_cod := 7;
    PROC2(p_sal => p_salario, p_dep => p_departamento, p_ID => p_cod);
    DBMS_OUTPUT.PUT_LINE('Salario: ' || p_salario);
    DBMS_OUTPUT.PUT_LINE('Departamento: ' || p_departamento);
END;
/

```

2.2 RECOMPILACIÓN Y ELIMINACIÓN DE PROCEDIMIENTOS

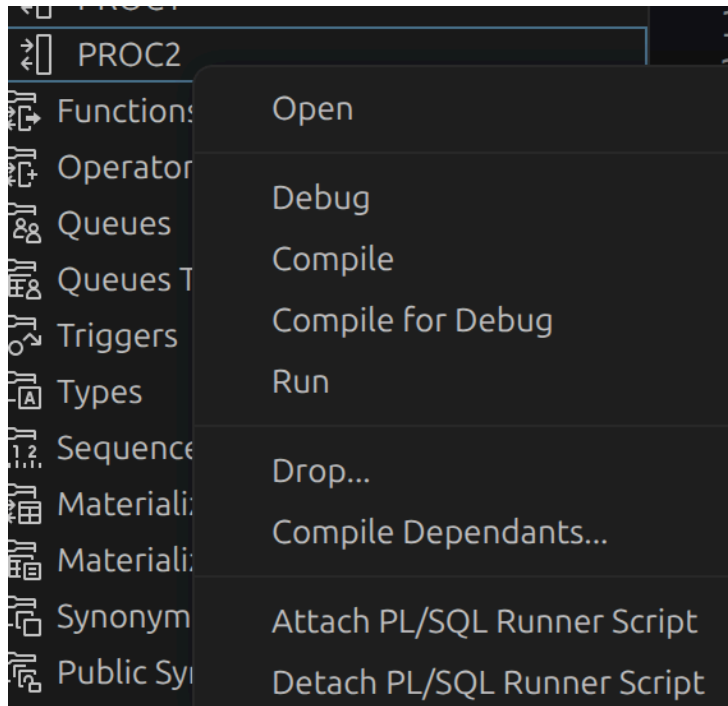
Para borrar un procedimiento utilizamos la instrucción DROP PROCEDURE:

```
DROP PROCEDURE nombre_procedimiento;
```

Para volver a compilar un procedimiento utilizamos ALTER (válido para procedimientos y funciones):

```
ALTER PROCEDURE|FUNCTION nombre_subprograma COMPILE;
```

Si estamos utilizando un entorno gráfico también tendremos las correspondientes opciones:



3.FUNCIONES

Una función es un bloque nombrado PL/SQL que devuelve un valor, y solamente tiene parámetros de entrada. Se almacena como un objeto en la base de datos para poder hacer repetidas ejecuciones del mismo.

Sintaxis:

```
CREATE [OR REPLACE] FUNCTION nombre [( parametro [, parametro] ...
)]
RETURN tipo_de_dato
IS|AS
[declaraciones_locales]
BEGIN
sentencias_ejecutables
[EXCEPTION manejadores_de_excepciones]
END [nombre];
```

Donde:

parametro := nombre_parametro [IN] tipo_dato [{:=|DEFAULT}
expresion]

RETURN tipo_dato representa el tipo de dato de devuelve la función.

En este caso los parámetros son siempre de entrada. Como IN es el valor por defecto se puede omitir.

Ejemplo:

```
CREATE OR REPLACE FUNCTION GET_SAL (p_ID empleado.codemp%TYPE)
RETURN NUMBER IS
    v_salario empleado.salemp%TYPE := 0;
BEGIN
    SELECT salemp INTO v_salario
    FROM empleado
    WHERE codemp = p_ID;
    RETURN v_salario;
END;
/
```

3.1 LLAMADAS A FUNCIONES

Igual que los procedimientos, las funciones están hechas para ser llamadas desde otros programa, sólo que en este caso la salida debe almacenarse en una variable del tipo que retorna la función:

```
DECLARE
    v_salario empleado.salemp%TYPE;
BEGIN
    v_salario := GET_SAL(&ID);
    DBMS_OUTPUT.PUT_LINE('Salario: ' || v_salario);
END;
/
```

También puede incluirse una llamada a una función dentro de una sentencia SQL:

```
SELECT GET_SAL(1) FROM DUAL;
```

Como en el caso de los procedimientos también podemos ejecutar funciones desde el propio cliente que estemos utilizando.

3.2 BORRADO DE FUNCIONES

Para borrar una función, de nuevo utilizamos la instrucción DROP con la siguiente sintaxis:

```
DROP FUNCTION nombre_función;
```

4. PAQUETES (PACKAGE)

Los **paquetes** en Oracle sirven para **agrupar y organizar funcionalidades** en una base de datos.

Son estructuras que agrupan objetos PL/SQL (funciones, procedimientos, tipos, etc.), lo que nos va a permitir tener programas estructurados agrupados por funcionalidades. Los paquetes están divididos en 2 partes: **especificación** (obligatoria) y **cuerpo** (no obligatoria).

La **especificación o encabezado** es la **interfaz entre el paquete y las aplicaciones** que lo utilizan, y es allí donde se **declaran los procedimientos y funciones** que podrán ser invocados desde fuera del paquete.

En el **cuerpo del paquete** se implementa la **especificación** del mismo. El cuerpo contiene los **detalles de implementación y declaraciones privadas**, manteniéndose todo esto oculto a las aplicaciones externas, siguiendo el conocido concepto de “caja negra”. Sólo las declaraciones hechas en la especificación del paquete son visibles y accesibles desde fuera del paquete (por otras aplicaciones o procedimientos almacenados) quedando los detalles de implementación del cuerpo del paquete totalmente ocultos e inaccesibles para el exterior.

Definimos la cabecera de nuestro paquete al que llamaré primer paquete:

```
CREATE OR REPLACE PACKAGE primer_paquete IS
-- Declaraciones
  PROCEDURE PROC2 (p_ID IN empleado.codemp%TYPE,
    p_sal OUT empleado.salemp%TYPE,
    p_dep OUT empleado.coddep%TYPE);

  FUNCTION GET_SAL (p_ID empleado.codemp%TYPE)
    RETURN NUMBER;
END;
/
```

A continuación definimos el cuerpo del paquete, incluyendo en este caso la lógica de procedimientos y funciones declarados en la cabecera:

```

CREATE OR REPLACE PACKAGE BODY primer_paquete IS
-- Bloques de código
    PROCEDURE PROC2 (p_ID IN empleado.codemp%TYPE,
        p_sal OUT empleado.salemp%TYPE,
        p_dep OUT empleado.coddep%TYPE) IS
    BEGIN
        SELECT salemp, coddep INTO p_sal, p_dep
        FROM empleado
        WHERE codemp = p_ID;
    END;

    FUNCTION GET_SAL (p_ID empleado.codemp%TYPE)
    RETURN NUMBER IS
        v_salario empleado.salemp%TYPE := 0;
    BEGIN
        SELECT salemp INTO v_salario
        FROM empleado
        WHERE codemp = p_ID;
        RETURN v_salario;
    END;
END;
/

```

Y para llamar a los subprogramas implementados en el paquete utilizamos la sintaxis:

```
nombre_paquete.nombre_procedimiento_o_función;
```

Ejemplo:

```
SELECT primer_paquete.get_sal(7) FROM DUAL;
```

4.1 VENTAJAS DEL USO DE PAQUETES

- Permite **modularizar** el diseño de nuestra aplicación: El uso de paquetes permite **encapsular** elementos relacionados entre sí (tipos, variables, procedimientos, funciones) en un único módulo PL/Sql que llevará un nombre que identifique la funcionalidad del conjunto.
- Otorga **flexibilidad** al momento de **diseñar la aplicación**: En el momento de diseñar una aplicación todo lo que necesitaremos inicialmente es la información de interfaz en la especificación del paquete. Puede codificarse y compilarse la especificación sin su cuerpo para posibilitar que otros subprogramas que referencian a estos elementos declarados puedan compilarse sin errores. De esta manera podremos armar un “**prototipo**” de nuestro sistema antes de codificar el detalle del mismo.

- Permite **ocultar los detalles de implementación**: Pueden especificarse cuáles tipos, variables y sub-programas dentro del paquete son públicos (visibles y accesibles por otras aplicaciones y sub-programas fuera del paquete) o privados (ocultos e inaccesibles fuera del paquete). Por ejemplo, dentro del paquete pueden existir procedimientos y funciones que serán invocados por otros programas, así como también otras rutinas de uso interno del paquete que no tendrán posibilidad de ser accedidas fuera del mismo. Esto asegura que cualquier cambio en la definición de estas rutinas internas afectará sólo al paquete donde se encuentran, simplificando el mantenimiento y protegiendo la integridad del conjunto.
- Agrega mayor **funcionalidad a nuestro desarrollo**: Las definiciones públicas de tipos, variables y cursores hechas en la especificación de un paquete persisten a lo largo de una sesión. Por lo tanto pueden ser compartidas por todos los subprogramas y/o paquetes que se ejecutan en ese entorno durante esa sesión. Por ejemplo, puede utilizarse esta técnica (en dónde sólo se define una especificación de paquete y no un cuerpo) para mantener tipos y variables globales a todo el sistema.
- Introduce **mejoras al rendimiento**: En relación a su ejecución, cuando un procedimiento o función que está definido dentro de un paquete es llamado por primera vez, todo el paquete es “subido” a memoria. Por lo tanto, posteriores llamadas al mismo u otros subprogramas dentro de ese paquete realizarán un acceso a memoria en lugar de a disco. Esto no sucede con procedimientos y funciones estándares.
- Permite la “**sobrecarga de funciones**” (**Overloading**): PL/SQL nos permite que varios procedimientos o funciones almacenadas, declaradas dentro de un mismo paquete, tengan el mismo nombre. Esto nos es muy útil cuando necesitamos que los sub-programas puedan aceptar parámetros que contengan diferentes tipos de datos en diferentes instancias. En este caso Oracle ejecutará la “versión” de la función o procedimiento cuyo encabezado se corresponda con la lista de parámetros recibidos.